

# HW 2-B: OWASP Juice Shop / OWASP Top 10

GitHub account: srijapentyala

## Part 1: Secure feature design

---

I ran Juice Shop locally and designed registration around three bugs I hit.

Login concatenates the email into SQL. I entered ' OR 1=1-- with a junk password and was signed in as admin@juice-sh.op. Juice Shop awarded Login Admin. The registration insert must use a parameterized query, so the email cannot change the statement.

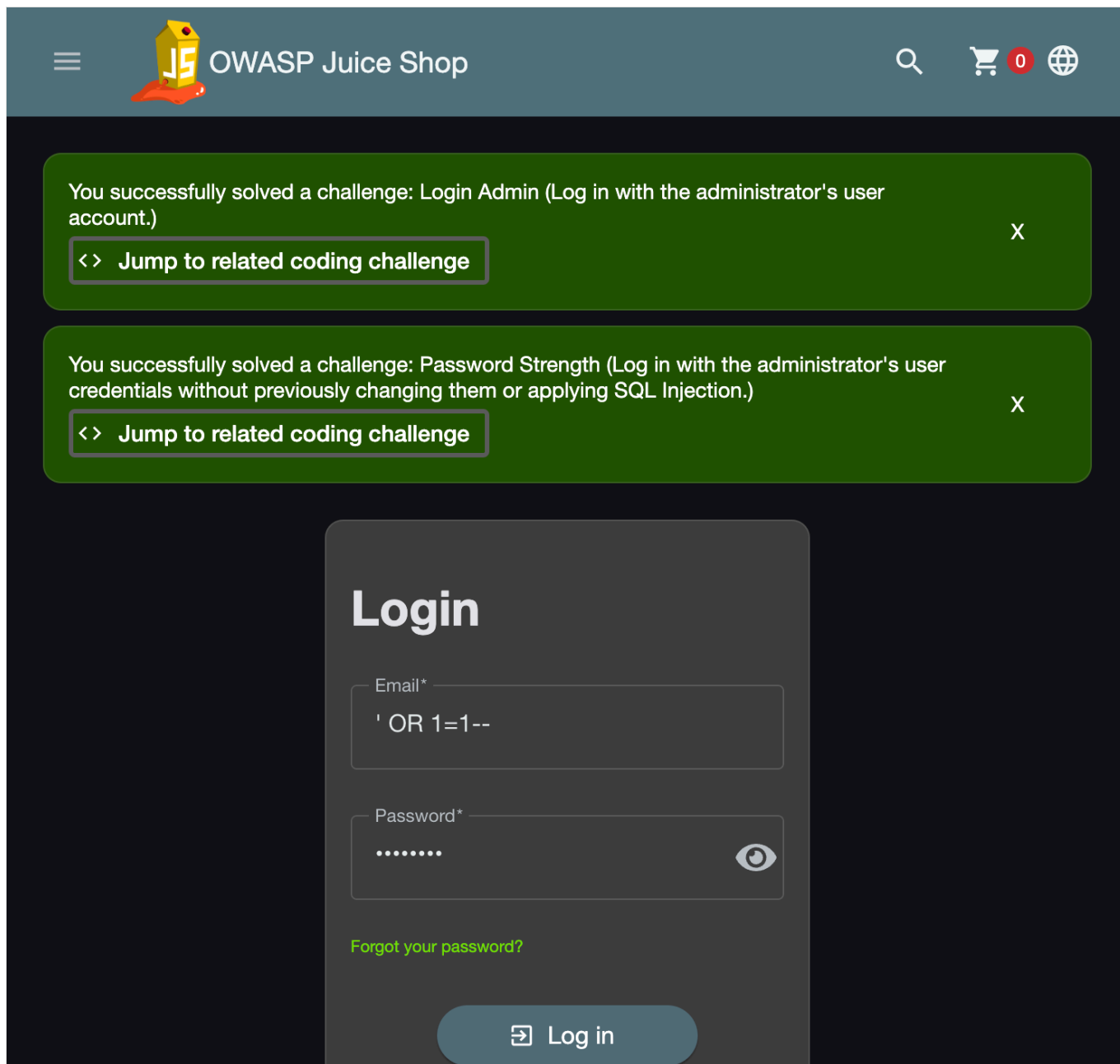


Figure 1. The injected email is still in the form. Juice Shop awarded Login Admin for that login, and Password Strength for the real administrator password admin123.

Search inserts the query as HTML. My term rendered as a yellow XSS heading, not as text. A registered username has to be encoded on output, and a Content-Security-Policy should block inline script.

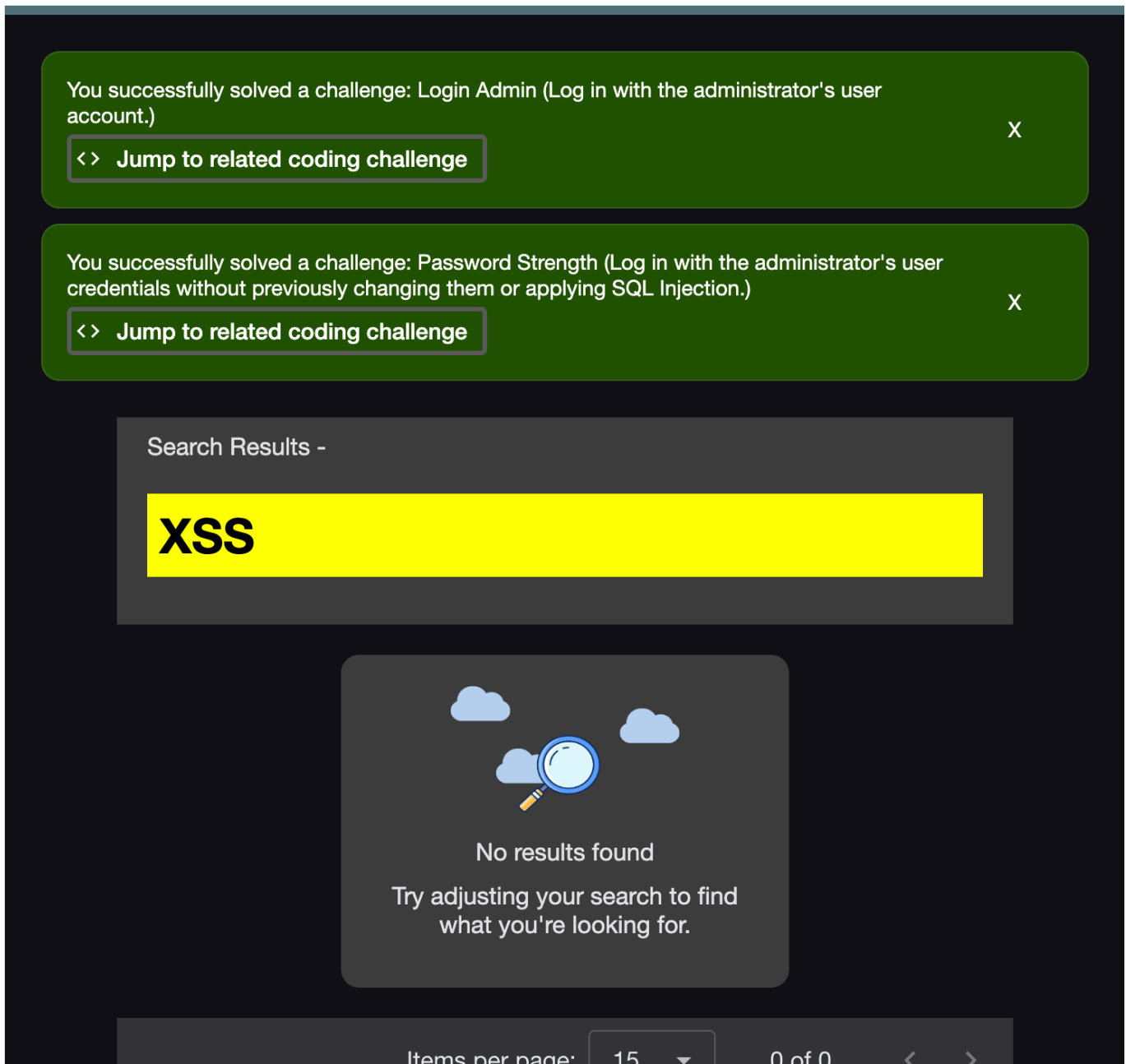


Figure 2. The search term was parsed as HTML. The tags never appear as text.

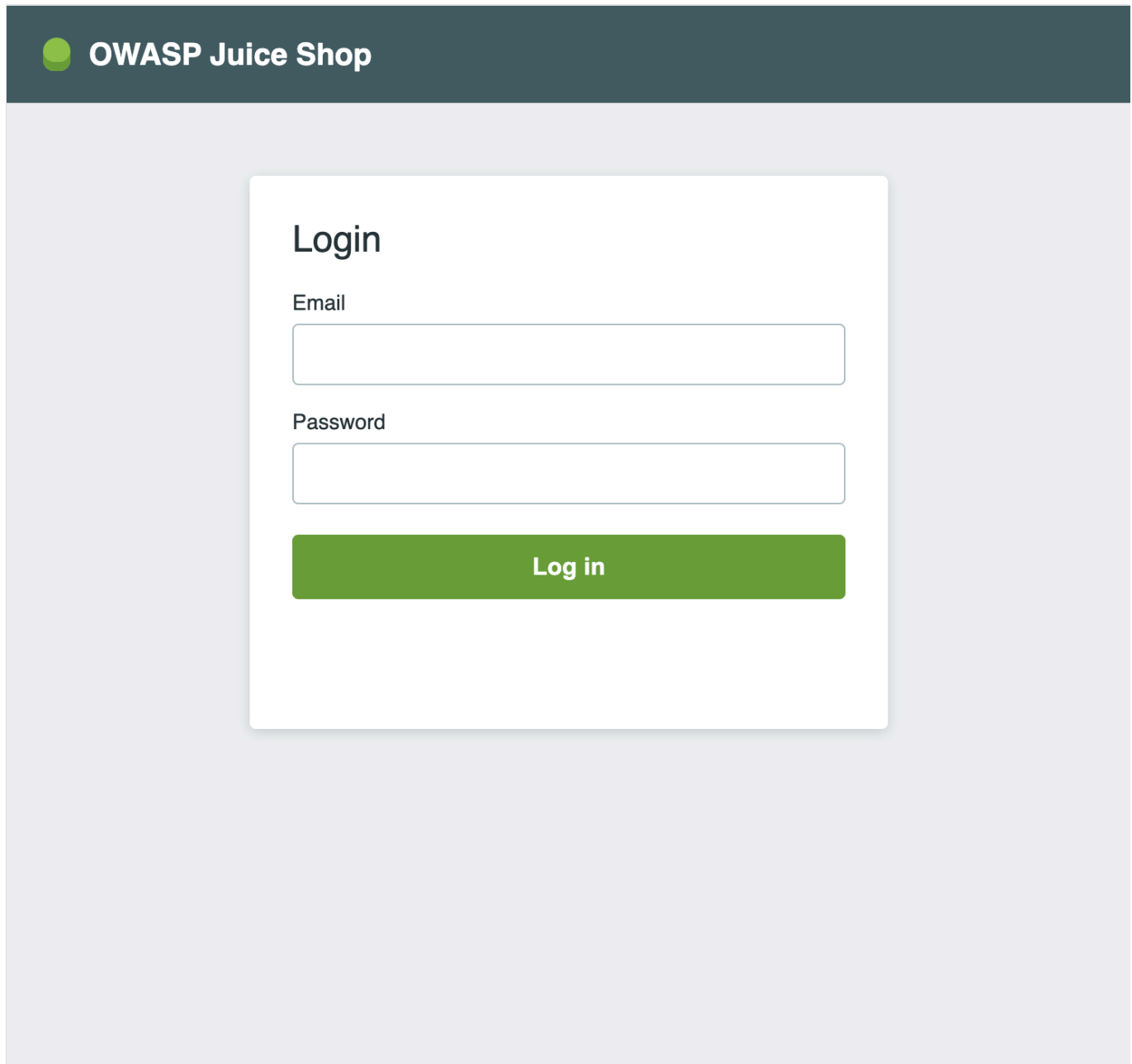
The administrator password is admin123, and the login token includes its MD5 hash. Password Strength solved only when I used that password, not injection. Registration should reject common passwords and store a bcrypt hash with a unique salt and cost 12. Never return the hash.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
bcrypt.checkpw(password.encode(), stored_hash)
```

## Part 2: Front-end login form

I built a login page with the same two fields as Juice Shop: email and password. login.js blocks an empty form, requires “@” in the email, and requires a password of at least 8 characters. server.py repeats those three checks, then compares the password with a PBKDF2 hash of the demo account. Input is not placed in a SQL string. Run `python3 server.py` and open `http://127.0.0.1:8080`. The demo account is `demo@juice-sh.op` / `JuiceShop1`.

Repository: <https://github.com/srijapentyala/hw2b-juice-shop-login>



The image shows a web browser window displaying the OWASP Juice Shop login page. The page has a dark blue header with the OWASP Juice Shop logo and name. Below the header is a light gray background. In the center, there is a white login form with a green border. The form has a title "Login" in bold black text. Below the title are two input fields: "Email" and "Password". The "Email" field is empty. The "Password" field is empty. Below the input fields is a green button with the text "Log in" in white. The form is centered on the page.

Figure 3. Login form before submission.

## Login

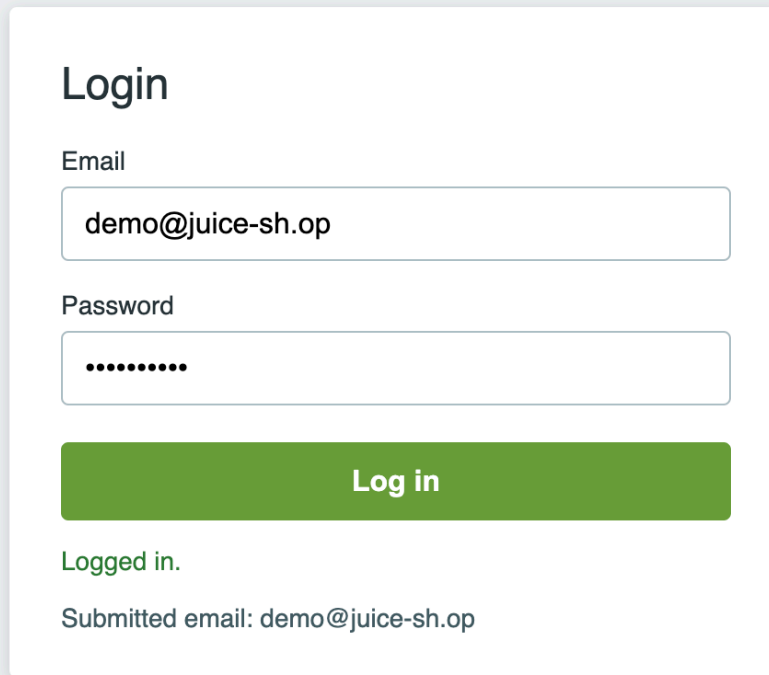
Email

Password

Log in

Email and password are required.

Figure 4. Client-side check rejects an empty submission.



Login

Email

demo@juice-sh.op

Password

.....

Log in

Logged in.

Submitted email: demo@juice-sh.op

Figure 5. Demo account passes both the browser check and the server check.

### Part 3: Trying to break the form

---

I attacked `http://127.0.0.1:8080` in two passes.

1. SQL injection. Email ' OR 1=1--@juice-sh.op, password password1, then Log in. The format checks passed. The server answered "Invalid email or password." There is no SQL string to rewrite, so the login failed.

## Login

Email

Password

**Log in**

Invalid email or password.

Submitted email: ' OR 1=1--@juice-sh.op

Figure 6. The SQL fragment is stored as an email address. Login fails.

2. XSS, which succeeds on the code in the repository. In `login.js` the result line is assigned with `innerHTML`. I entered `<img src=x onerror="document.getElementById('message').textContent='JavaScript executed'">@a.co` and password `password1`, then clicked Log in. The image could not load, the handler ran, and the status text became "JavaScript executed." Repeat it with `python3 server.py` and open `http://127.0.0.1:8080`.

## Login

Email

Password

Log in

JavaScript executed

Submitted email: @a.co

Figure 7. The status line was written by the image handler, not by the login code.

Fix: assign that line with `textContent`, and send `Content-Security-Policy: script-src 'self'` so an inline handler cannot run.